

KRO-nicles of Kubernetes

Taming Resources the Open Source Way

Hari Charan Ayada

Senior Solutions Architect



Kubernetes for platform building

Declarative configuration, continuous reconciliation

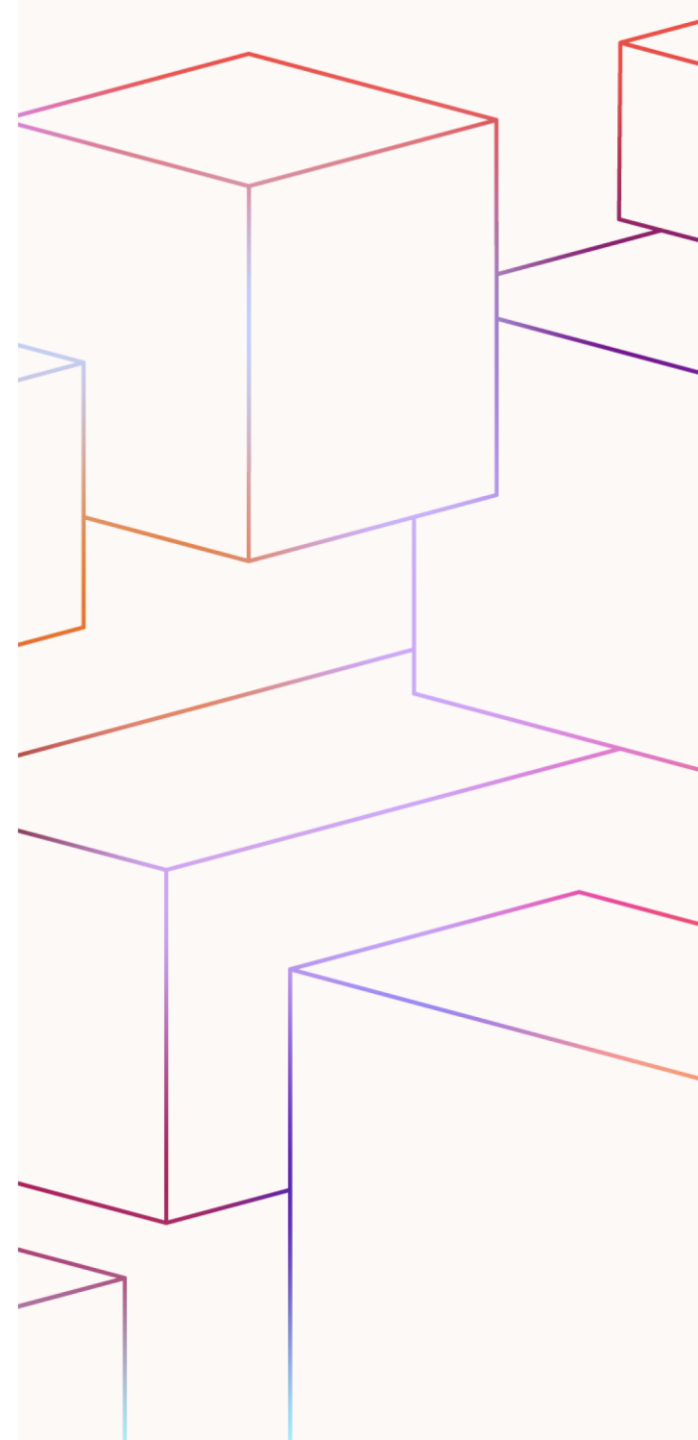
Kubernetes design principles facilitate a transparent adoption of DevOps best practices

Extensibility through the Operator Pattern

Custom Resource Definitions and controllers extend Kubernetes capabilities

Kubernetes is ideal for modern platform building

Majority of Kubernetes enterprise adopters have established platform engineering teams



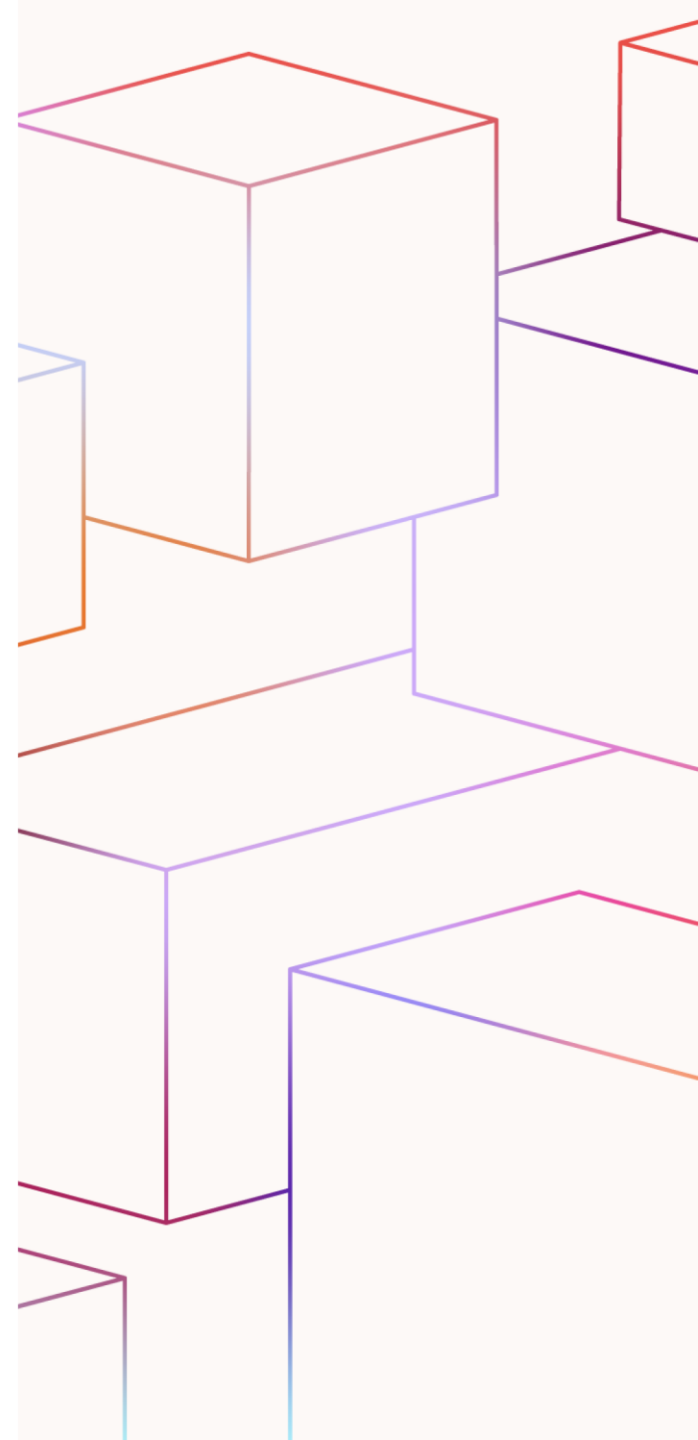
Kubernetes complexity challenge

Developer challenges

- Kubernetes learning curve, 300+ level is steep
- Understanding resource relationships
- Managing resources and applications together
- Yaml-plexity.

Platform challenges

- Standardize and simplify, with minimal constraints
Extend to meet requirements, but on rails
- Establish and maintain organizational standards
- Tame resource complexity

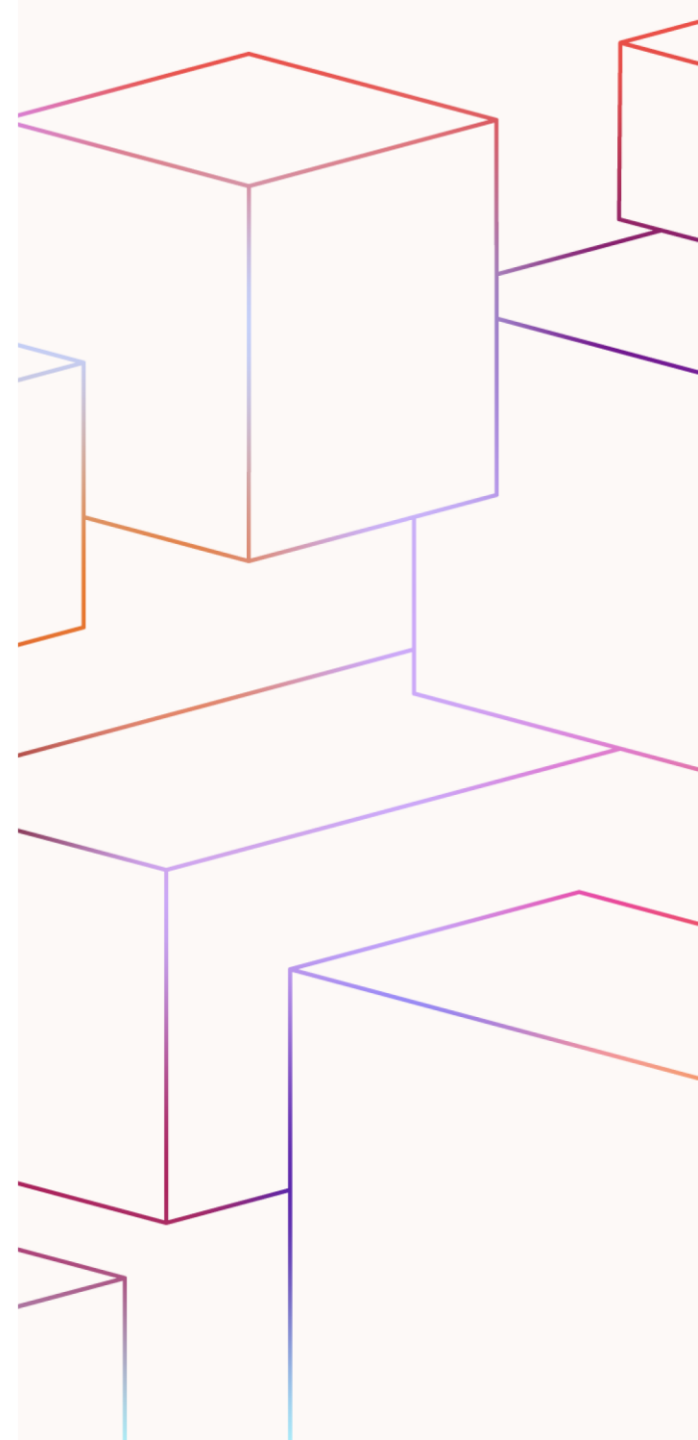


Kubernetes is endlessly customizable!

Custom controllers

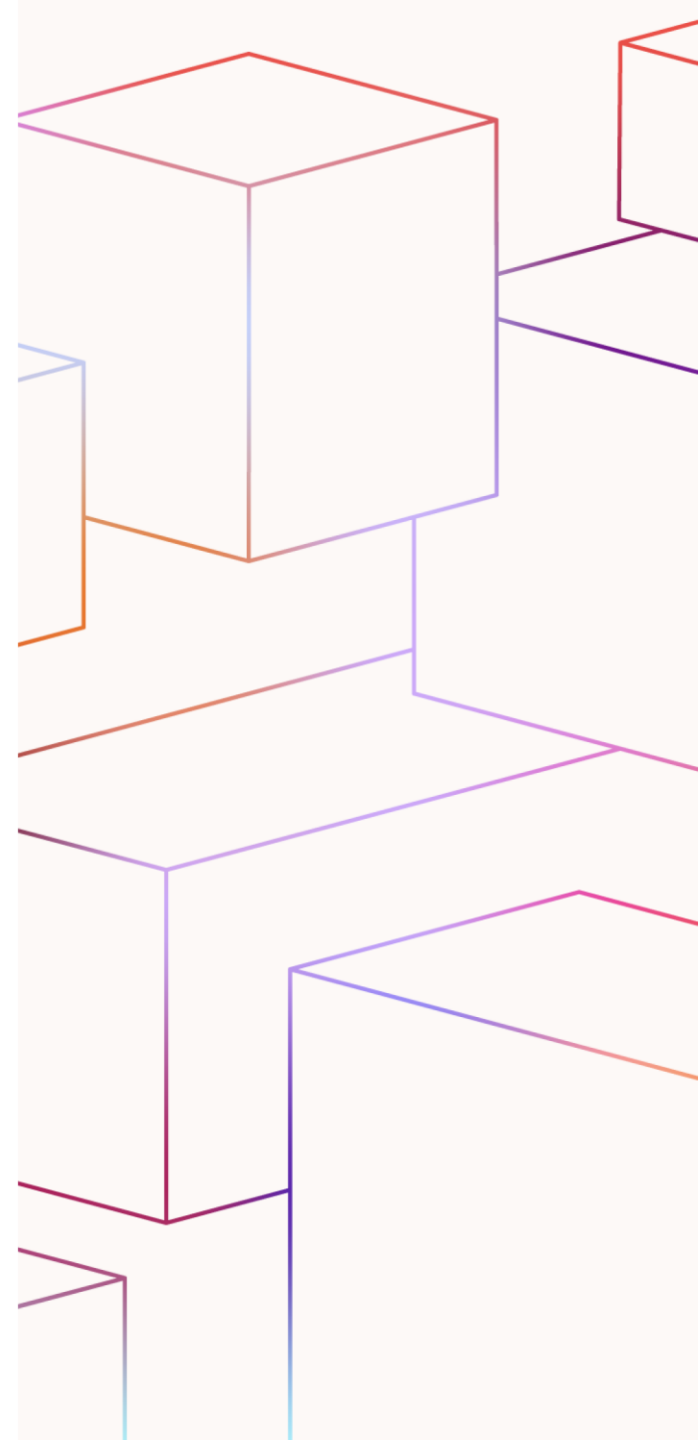
- Requires specialized knowledge and skills to build and maintain
- Ongoing investment, more moving parts
- Often built to orchestrate configuration and management of other resources
- Custom controllers that don't implement differentiated business logic are effectively technical debt

Kubernetes could use a new feature.



Kubernetes is endlessly customizable!

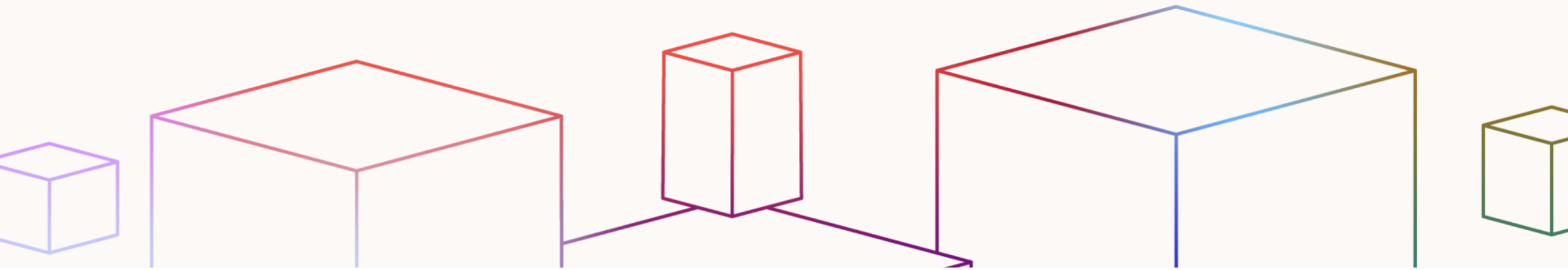
Lets see in action!



Kube Resource Orchestrator (KRO)

- A Kubernetes native and structured framework for resource group orchestration
- Abstracts and automates native Kubernetes extension and customization
- Provides a simplified in-cluster experience for end users
- Enables standardized deployments across environments
- Developed in collaboration by AWS, Azure, and Google Cloud

kro.run
github.com/kro-run/kro
1.9k stars
45 contributors
Maintainers from AWS, GCP, Azure



Platform building with kro

Author and distribute platform building blocks

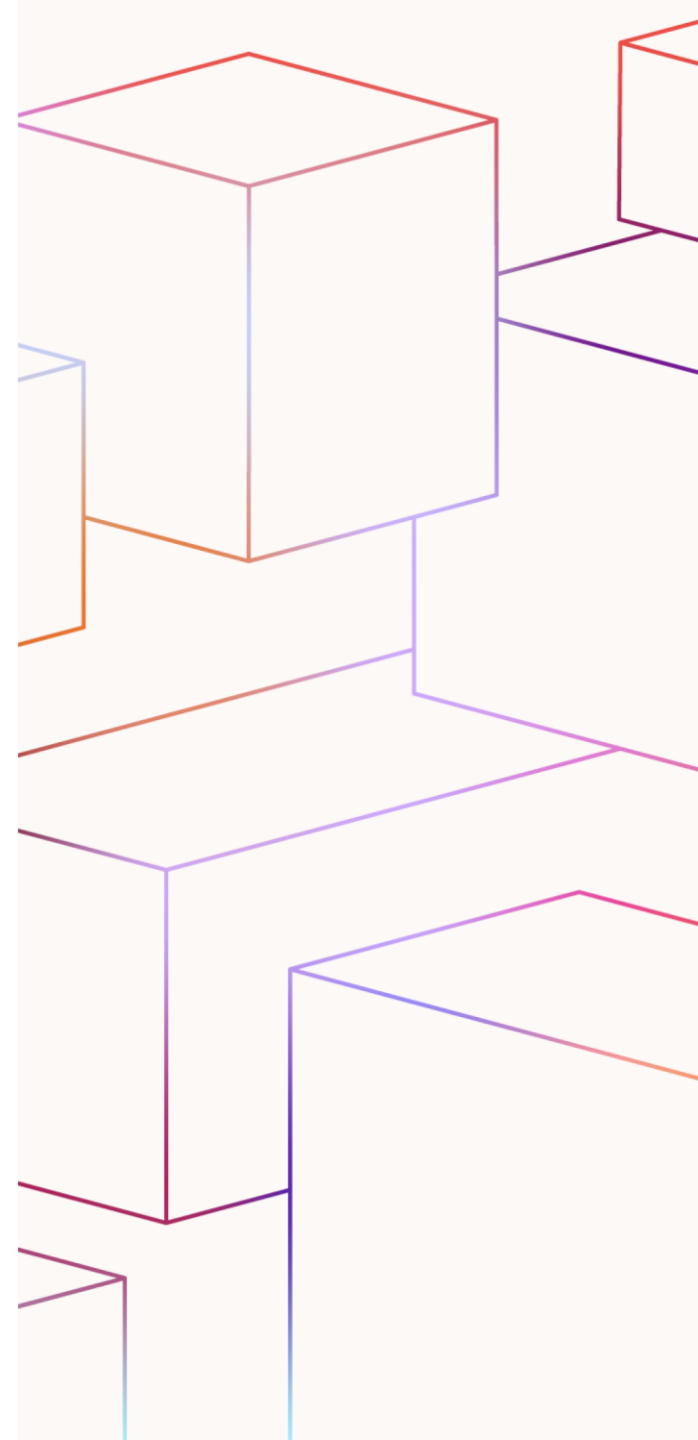
High-level abstractions for complex resource configurations, with optionally hard-wired config

Unify cloud and KBs resource management

Dynamic resource configuration and management of cloud resources

Simplify CRD authoring

Define your CRD's with Simple Schema, human-readable and easy to reason about



Platform building with kro

Author and distribute platform building blocks

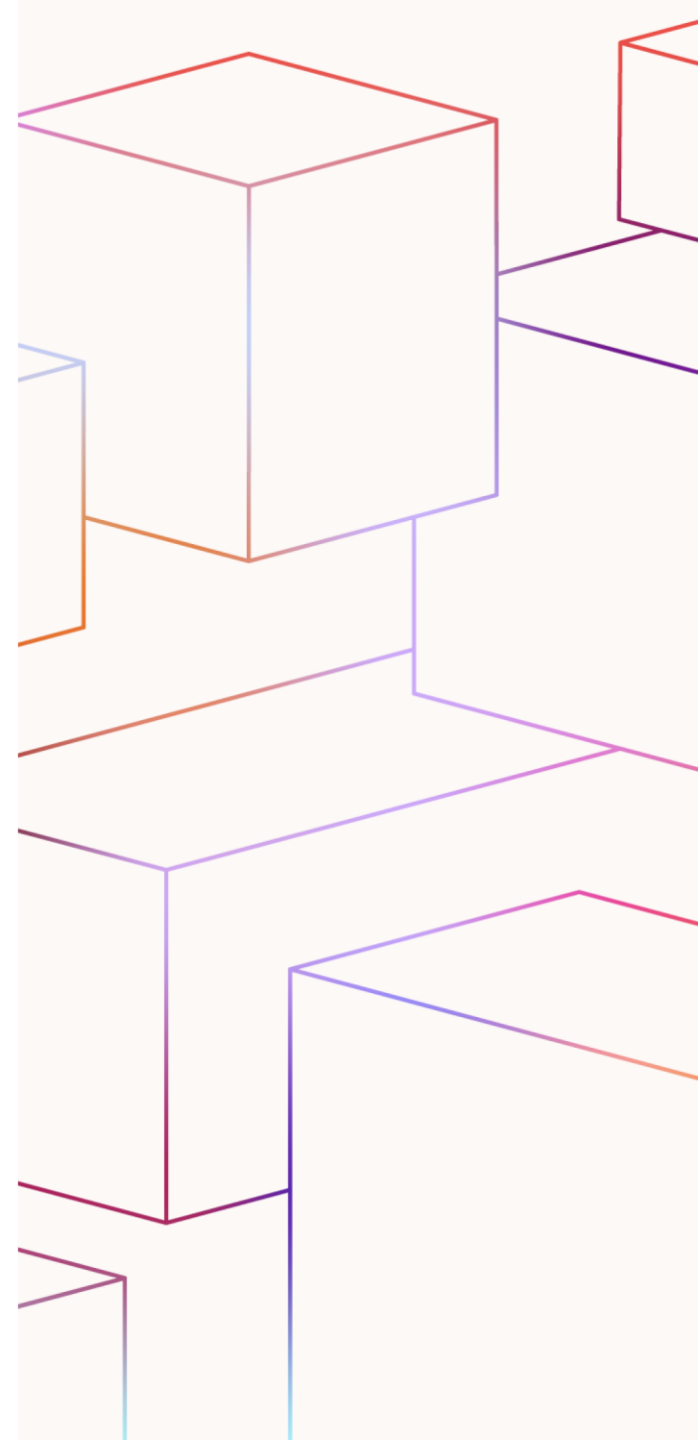
High-level abstractions for complex resource configurations, with optionally hard-wired config

Unify cloud and KBs resource management

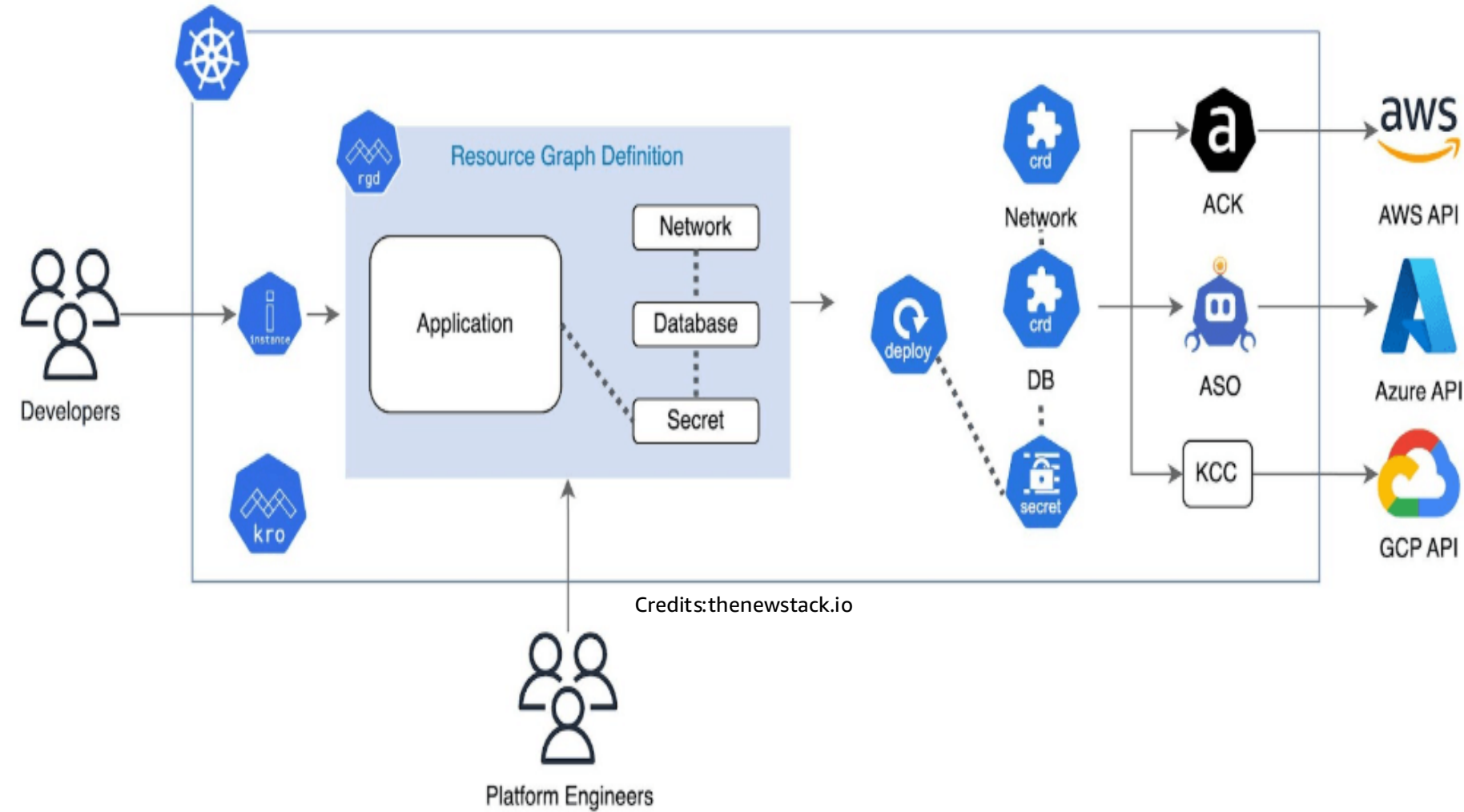
Dynamic resource configuration and management of cloud resources

Simplify CRD authoring

Define your CRD's with Simple Schema, human-readable and easy to reason about



Platform building with kro



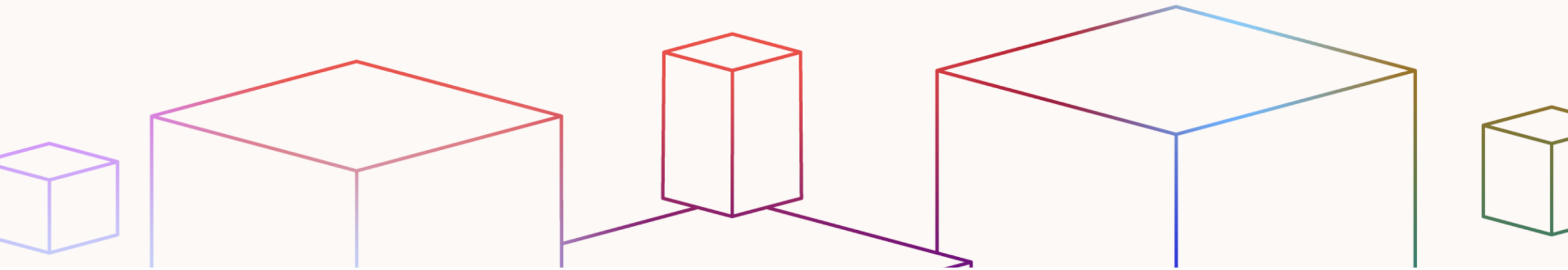
Core concepts

Custom Resource Definitions simplified

- Dynamically creates and registers CRDs for you, vastly simplifying authoring
- Single controller adapts to support your custom resources, simplifying operations
- Detects CRD breaking changes ahead of big bang surprises in your cluster
- Protects against backward incompatibility

OpenAPI compatibility

- All wrapped up with Simple Schema, reducing YAML engineering
- Human-friendly schema format that compiles to OpenAPI Spec
- Easy to maintain and reason about over time, no matter how complex your resources



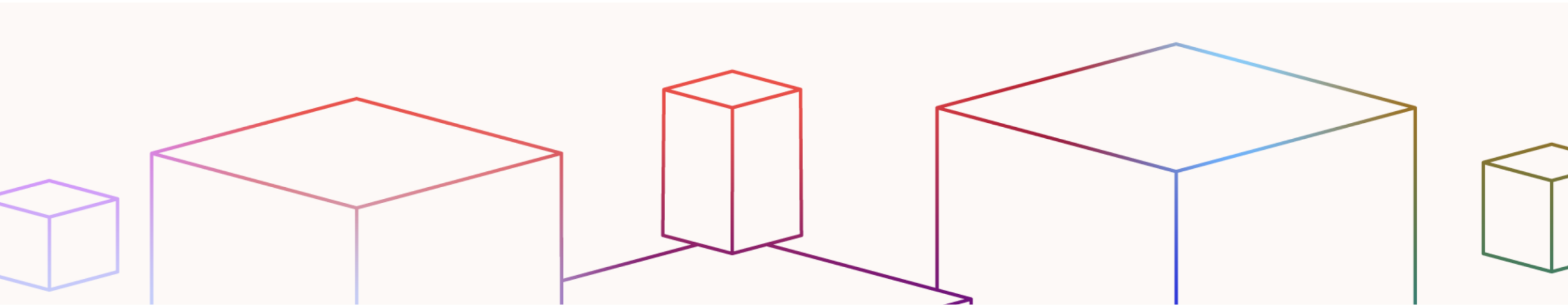
Core concepts

Structured approach vs templating

- No functions or complex programming constructs, no for loops (Non-Turing complete technology)
- Expressive enough for most scenarios

Automatic dependency management

- Infers resource ordering, ensuring proper creation and deletion sequencing
- No need for explicit dependsOn declarations (though this may be useful in some cases)
- Safe cascaded deletions



```

apiVersion: kro.run/v1alpha1
kind: ResourceGraphDefinition
metadata:
  name: ack-s3-demo
spec:
  schema:
    apiVersion: v1alpha1
    kind: S3Demo
    spec:
      bucketName: "string | required=true"
  resources:
    - id: bucket
      template:
        apiVersion: s3.services.k8s.aws/v1alpha1
        kind: Bucket
        metadata:
          name: ${schema.spec.bucketName}
        spec:
          name: ${schema.spec.bucketName}

```

AWS S3

GCP Bucket

```

apiVersion: kro.run/v1alpha1
kind: ResourceGraphDefinition
metadata:
  name: kcc-gcs-demo
spec:
  schema:
    apiVersion: v1alpha1
    kind: GcsDemo
    spec:
      bucketName: "string | required=true"
  resources:
    - id: gcs
      template:
        apiVersion: storage.cnrm.cloud.google.com/v1beta1
        kind: StorageBucket
        metadata:
          name: ${schema.spec.bucketName}
        spec:
          location: US
          bucketPolicyOnly: true

```

Crossplane PGSql

```

apiVersion: kro.run/v1alpha1
kind: ResourceGraphDefinition
metadata:
  name: xp-postgres-demo
spec:
  schema:
    apiVersion: v1alpha1
    kind: DbDemo
    spec:
      engineVersion: "string | default=13"
  resources:
    - id: db
      template:
        apiVersion: database.aws.crossplane.io/v1beta1
        kind: RDSInstance
        metadata:
          name: demo-db
        spec:
          forProvider:
            engine: postgres
            engineVersion: ${schema.spec.engineVersion}
            instanceClass: db.t3.micro
            writeConnectionSecretToRef:
              name: db-conn
              namespace: default

```

Certmanger

```

apiVersion: kro.run/v1alpha1
kind: ResourceGraphDefinition
metadata:
  name: certmgr-demo
spec:
  schema:
    apiVersion: v1alpha1
    kind: CertDemo
    spec:
      domain: "string | required=true"
  resources:
    - id: cert
      template:
        apiVersion: cert-manager.io/v1
        kind: Certificate
        metadata:
          name: ${schema.spec.domain}-crt
        spec:
          dnsNames:
            - ${schema.spec.domain}
          issuerRef:
            name: letsencrypt
            kind: ClusterIssuer

```

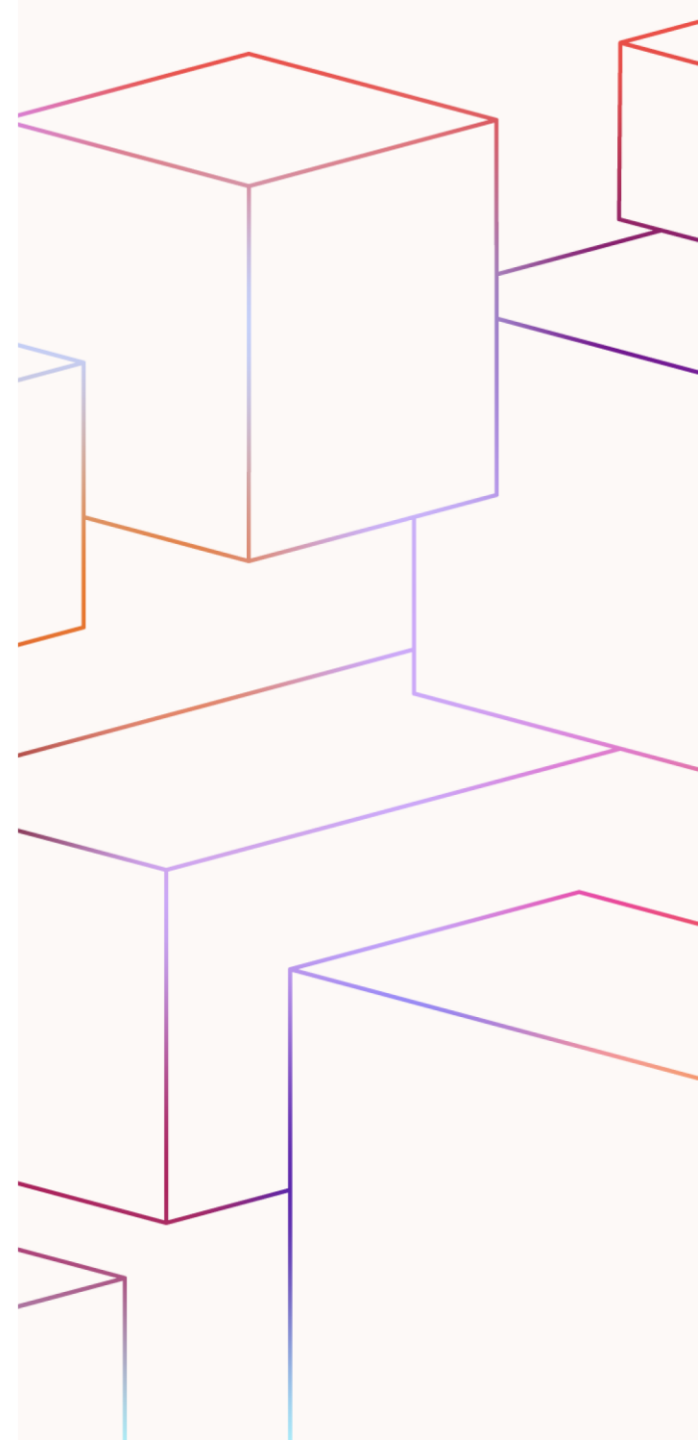


What's next?

It's very early days for Kro, but there is a lot of interest
We are working on the next features in upcoming project milestones:

- 0.3 External references
- 0.4 Collections

We are on our way to code complete for alpha, and would love to hear from you



Thank you!

Hari Charan Ayada
Solutions Architect

